

LAB PRAOGRAMS

Name: M.MANASA

Reg no:192425189

Course code: MLA0201

Course: FUNDAMENTALS OF MACHINE LEARNING

1}

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
data = [  
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes'],  
    ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes'],  
    ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No'],  
    ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']  
]
```

```
h = ['0', '0', '0', '0', '0', '0']
```

```
for row in data:
```

```
    if row[6] == 'Yes':
```

```
        for i in range(6):
```

```
            if h[i] == '0':
```

```
                h[i] = row[i]
```

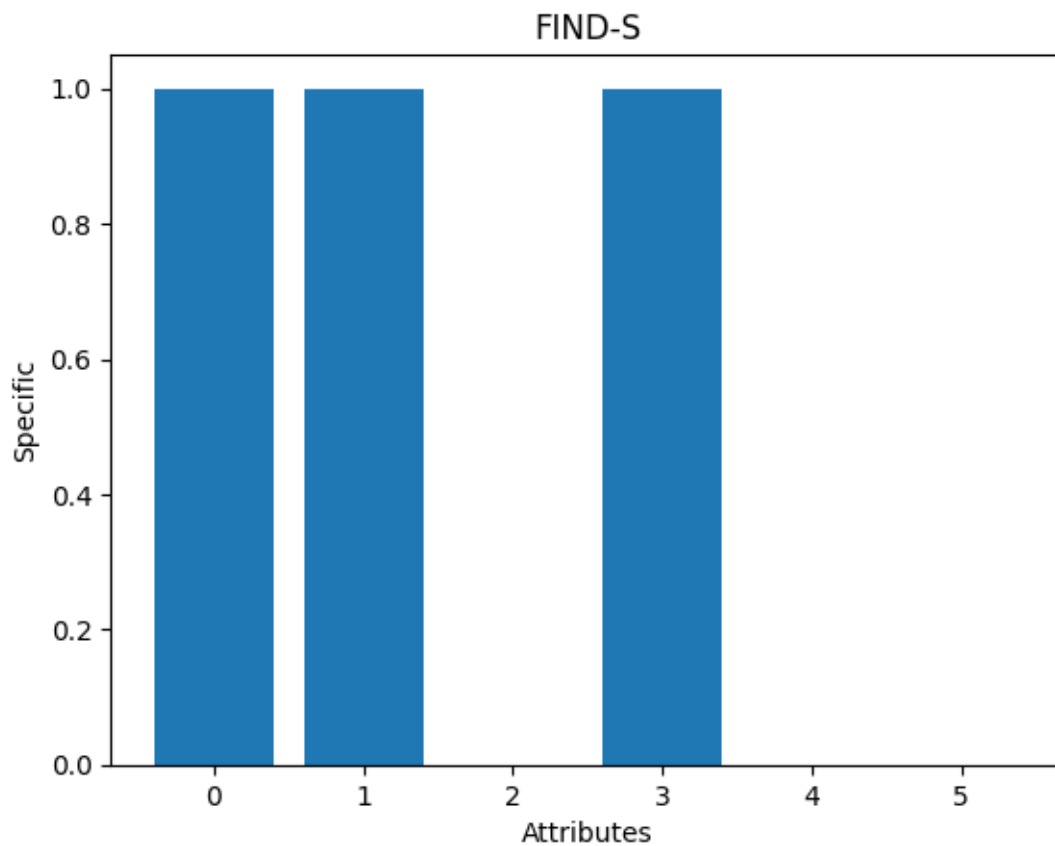
```
            elif h[i] != row[i]:
```

```
                h[i] = '?'
```

```
print("Hypothesis:", h)
```

```
plt.bar(range(6), [1 if x != '?' else 0 for x in h])
plt.xlabel("Attributes")
plt.ylabel("Specific")
plt.title("FIND-S")
plt.show()
```

Hypothesis: ['Sunny', 'Warm', '?', 'Strong', '?', '?']



2}

```
import matplotlib.pyplot as plt
```

```
data = [
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']
```

```
]
```

```
S = ['0', '0', '0', '0', '0', '0']
```

```
for row in data:
```

```
    if row[6] == 'Yes':
```

```
        for i in range(6):
```

```
            if S[i] == '0':
```

```
                S[i] = row[i]
```

```
            elif S[i] != row[i]:
```

```
                S[i] = '?'
```

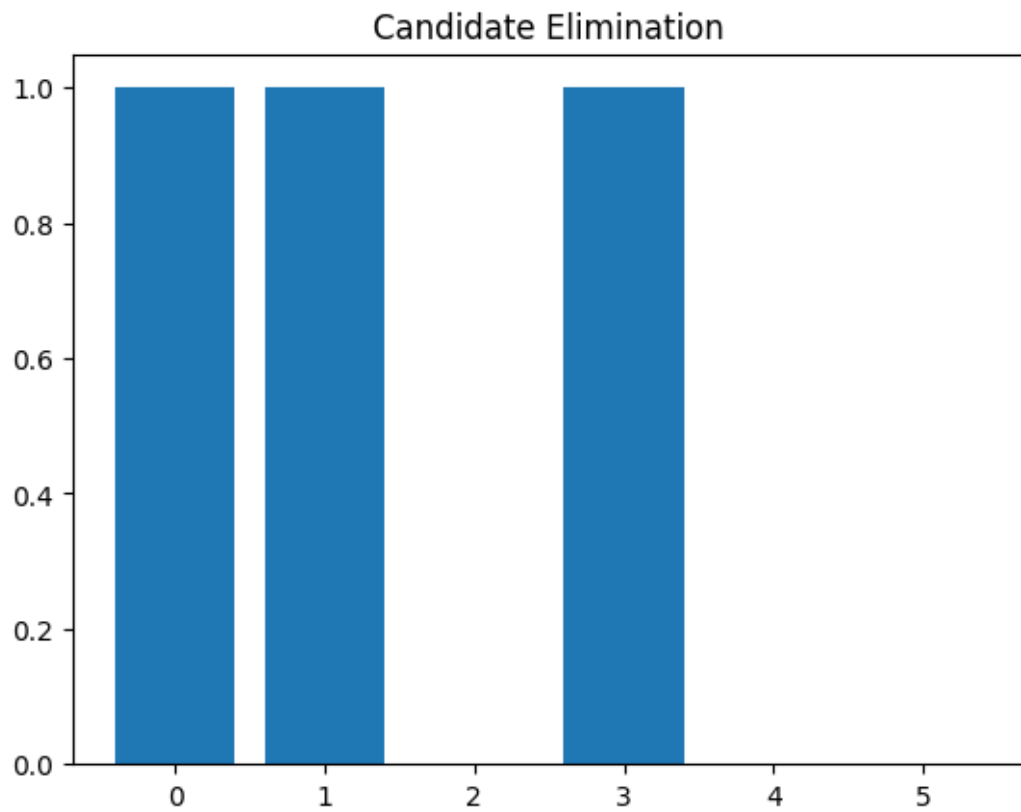
```
print("Specific Boundary:", S)
```

```
plt.bar(range(6), [1 if x != '?' else 0 for x in S])
```

```
plt.title("Candidate Elimination")
```

```
plt.show()
```

```
Specific Boundary: ['Sunny', 'Warm', '?', 'Strong', '?', '?']
```



```
3}
```

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
```

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

```
model = DecisionTreeClassifier(criterion='entropy', random_state=1)
```

```
model.fit(X, y)
```

```
prediction = model.predict([[5.1, 3.5, 1.4, 0.2]])
```

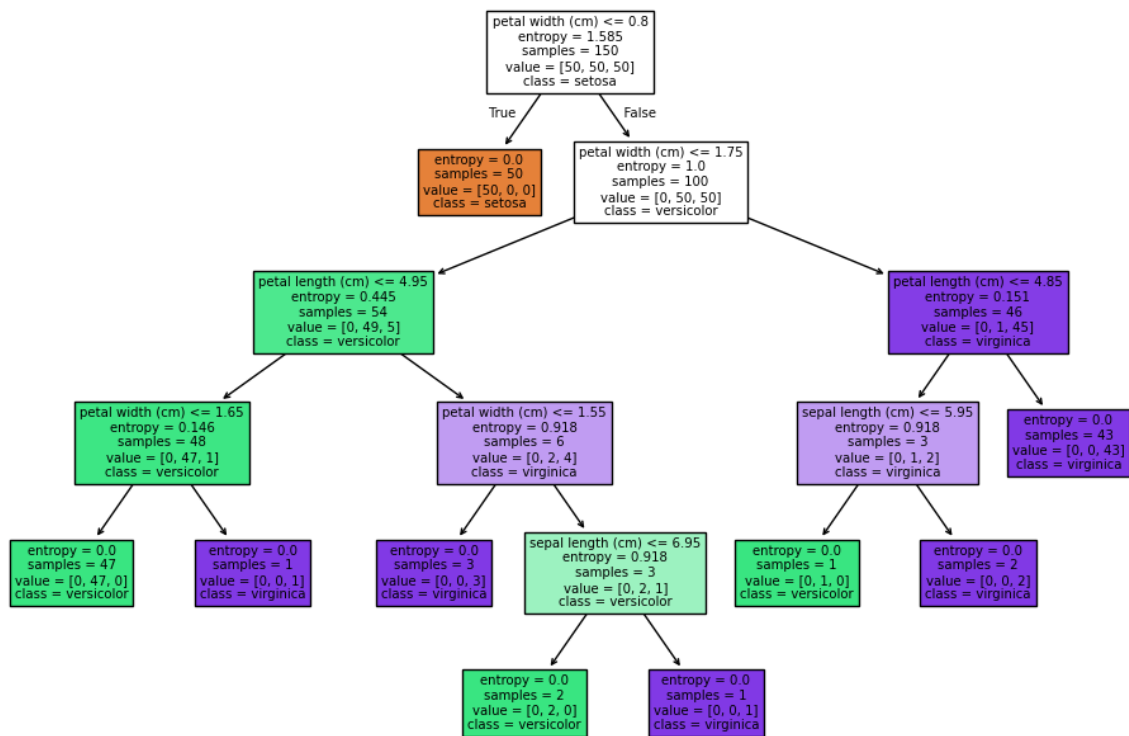
```
print("Predicted Class:", iris.target_names[prediction][0])
```

```
plt.figure(figsize=(12, 8))

plot_tree(model, feature_names=iris.feature_names,
          class_names=iris.target_names, filled=True)

plt.show()

Predicted Class: setosa
```



```
4}

from sklearn.neural_network import MLPClassifier

import matplotlib.pyplot as plt

X = [[0,0], [0,1], [1,0], [1,1]]
y = [0, 1, 1, 0]

model = MLPClassifier(hidden_layer_sizes=(4,),
                      activation='relu',
                      max_iter=2000,
```

```
random_state=1)
```

```
model.fit(X, y)
```

```
print("Predictions:")
```

```
print(model.predict(X))
```

```
plt.plot(model.loss_curve_)
```

```
plt.xlabel("Iterations")
```

```
plt.ylabel("Loss")
```

```
plt.title("Backpropagation Training")
```

```
plt.show()
```

```
/usr/local/lib/python3.13/dist-
```

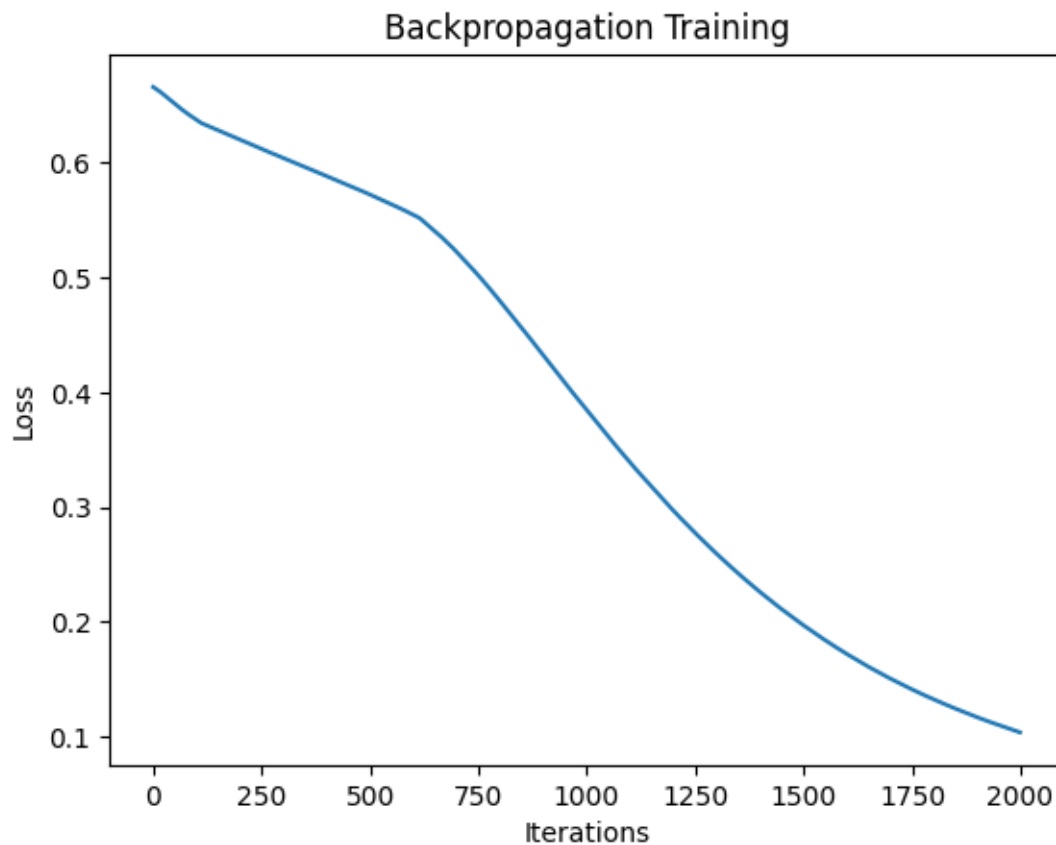
```
packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning:  
Stochastic Optimizer: Maximum iterations (2000) reached and the optimization hasn't  
converged yet.
```

```
warnings.warn(  

```

```
Predictions:
```

```
[0 1 1 0]
```



5}

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
```

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1)
```

```
model = KNeighborsClassifier(n_neighbors=5)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
plt.scatter(X_test[:,0], X_test[:,1], c=y_pred)
```

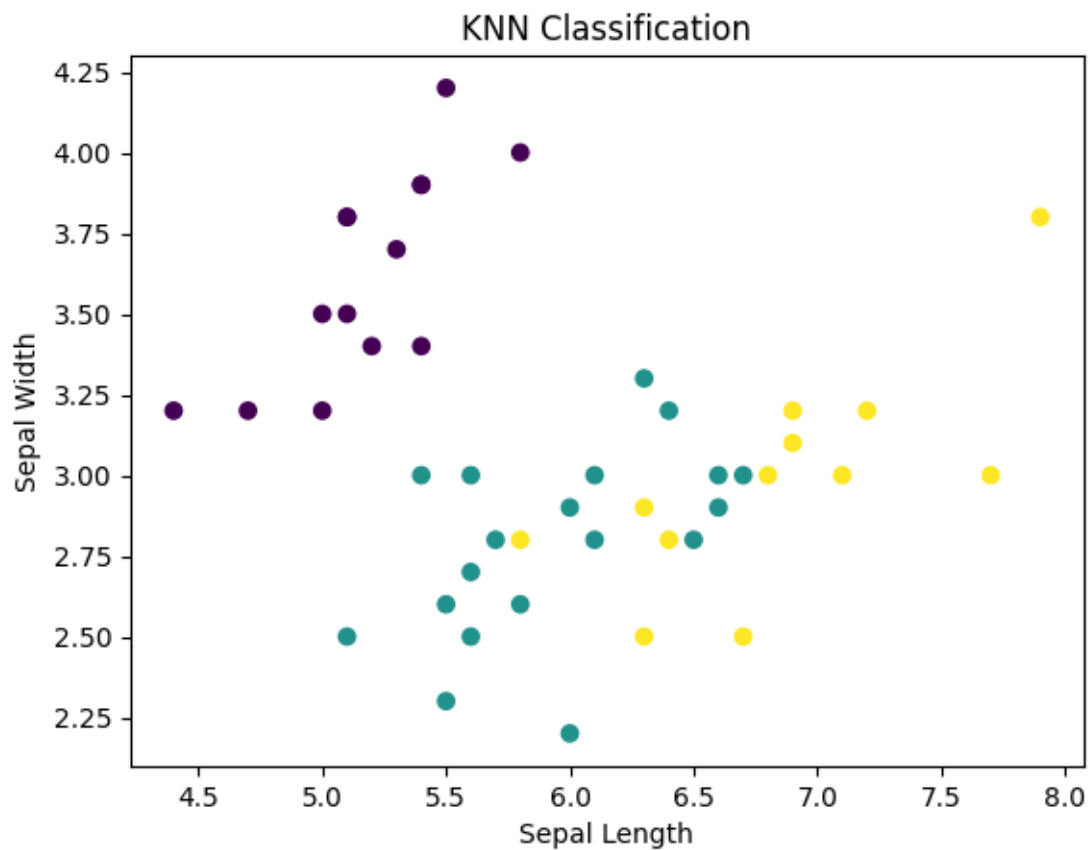
```
plt.xlabel("Sepal Length")
```

```
plt.ylabel("Sepal Width")
```

```
plt.title("KNN Classification")
```

```
plt.show()
```

```
Accuracy: 0.9777777777777777
```



```
6}
```

```
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
```



```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
```

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)
```

```
model = GaussianNB()
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
print("Confusion Matrix:")
```

```
print(cm)
```

```
ConfusionMatrixDisplay(cm).plot()
```

```
plt.title("Naive Bayes Confusion Matrix")
```

```
plt.show()
```

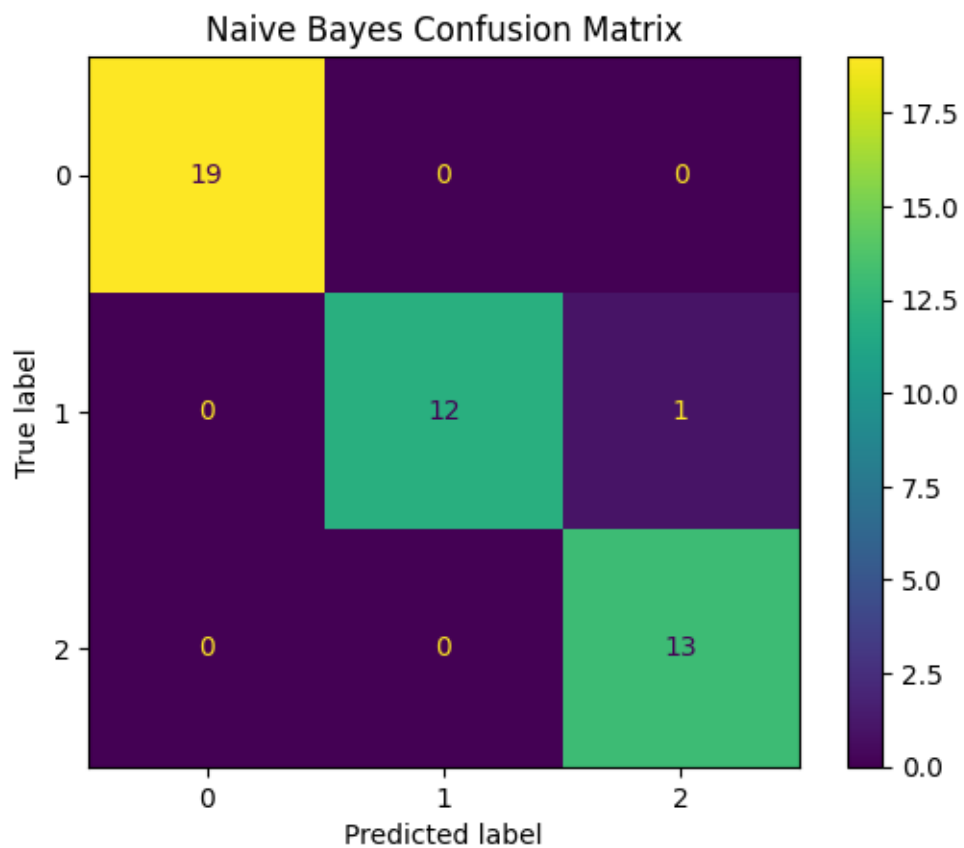
```
Accuracy: 0.9777777777777777
```

```
Confusion Matrix:
```

```
[[19 0 0]
```

```
 [ 0 12 1]
```

```
[ 0  0 13]]
```



```
7}
```

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
```

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1)
```

```

model = LogisticRegression(max_iter=200)

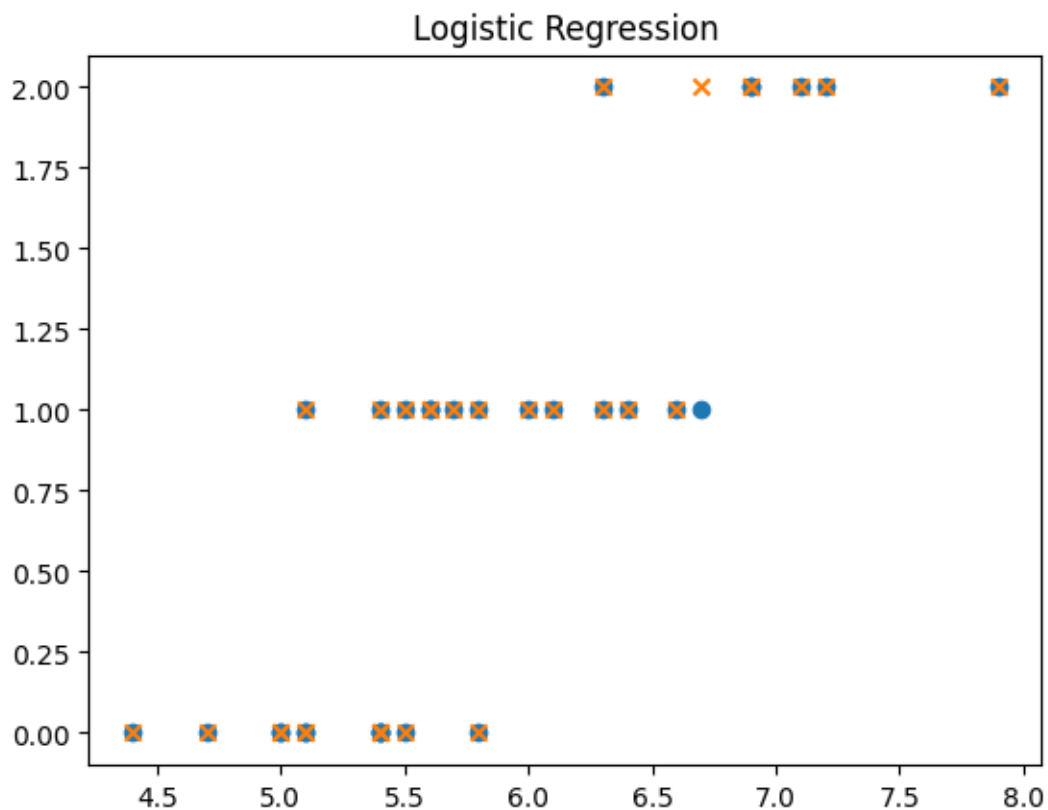
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

plt.scatter(X_test[:,0], y_test)
plt.scatter(X_test[:,0], y_pred, marker='x')
plt.title("Logistic Regression")
plt.show()
Accuracy: 0.9666666666666667

```



```

8}

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

```

```
X = np.array([[1], [2], [3], [4], [5]])
```

```
y = np.array([2, 4, 6, 8, 10])
```

```
model = LinearRegression()
```

```
model.fit(X, y)
```

```
prediction = model.predict([[6]])
```

```
print("Prediction for 6:", prediction[0])
```

```
plt.scatter(X, y)
```

```
plt.plot(X, model.predict(X))
```

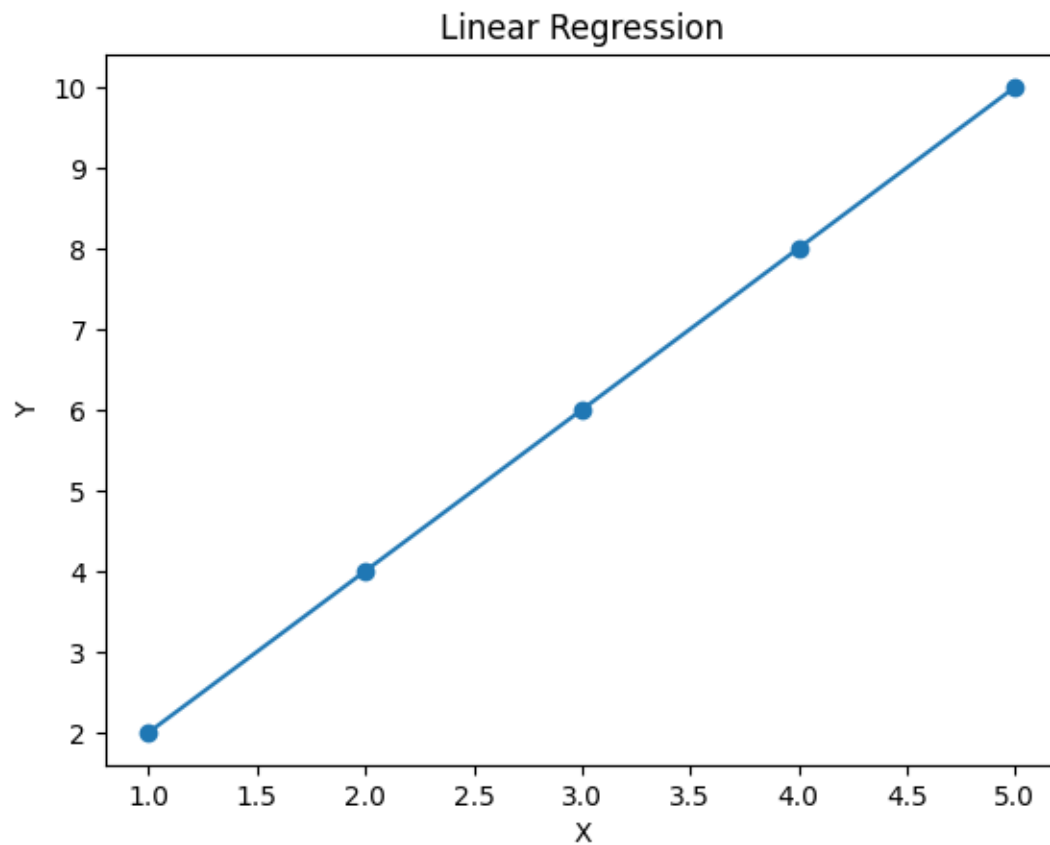
```
plt.xlabel("X")
```

```
plt.ylabel("Y")
```

```
plt.title("Linear Regression")
```

```
plt.show()
```

Prediction for 6: 12.0



```
9import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
```

```
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([1, 4, 9, 16, 25])
```

```
# Linear Regression
linear = LinearRegression()
linear.fit(X, y)
```

```
# Polynomial Regression
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)
```

```
polynomial = LinearRegression()
polynomial.fit(X_poly, y)

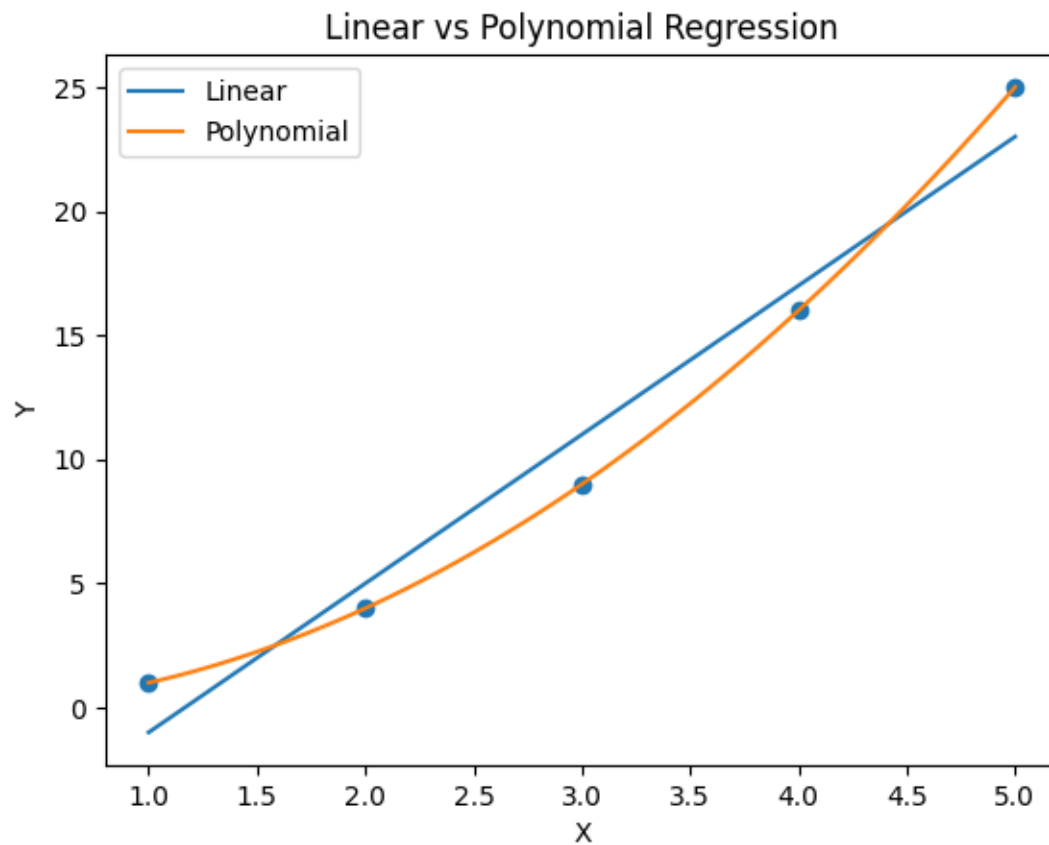
# Predictions
X_new = np.linspace(1, 5, 100).reshape(-1, 1)

y_linear = linear.predict(X_new)
y_poly = polynomial.predict(poly.transform(X_new))

print("Linear Prediction:", linear.predict([[6]])[0])
print("Polynomial Prediction:", polynomial.predict(poly.transform([[6]])[0])

plt.scatter(X, y)
plt.plot(X_new, y_linear, label="Linear")
plt.plot(X_new, y_poly, label="Polynomial")
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Linear vs Polynomial Regression")
plt.legend()
plt.show()
}

Linear Prediction: 28.999999999999996
Polynomial Prediction: 36.000000000000002
```



10}

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.mixture import GaussianMixture
```

```
X = np.array([
    [1, 2], [1, 3], [2, 2], [2, 3],
    [8, 8], [9, 8], [8, 9], [9, 9]
])
```

```
model = GaussianMixture(n_components=2, random_state=1)
```

```
model.fit(X)
```

```
labels = model.predict(X)
```

```
print("Cluster Labels:")
```

```
print(labels)
```

```
plt.scatter(X[:,0], X[:,1], c=labels)
```

```
plt.xlabel("X")
```

```
plt.ylabel("Y")
```

```
plt.title("Expectation-Maximization using GMM")
```

```
plt.show()
```

Cluster Labels:

[0 0 0 1 1 1 1]

